

PART I - Computer Science 1 (CS1)

Chapter 1

Common CS1 Activities and Lessons

April Crockett¹, Jane Doe²

¹Tennessee Technological University, acrockett@tntech.edu

²XYZ University

Abstract (Sushil)

This chapter distills the common CS1 activities, implementation strategies, challenges, and lessons learned across the eight institutional CS1 exemplars in this volume. It serves as a shared resource chapter between the project overview in Chapter 0 and the institution-specific CS1 chapters 2-9 that follow. Rather than presenting another local course implementation, the chapter identifies recurring activity families and reusable guidance that cut across TNTECH, Knox, USI, UNL, Webster, Casper, HPU, and MSU. Its purpose is to help instructors locate activities that can be adopted as a single class activity, a short module, a laboratory exercise, or part of a broader course-level infusion of parallel and distributed computing (PDC) into CS1, all the while ensuring that the basic CS1 learning outcomes are preserved.

The activities summarized here introduce accessible versions of core PDC ideas, including data parallelism, task decomposition, speedup, load balance, contention, communication, distributed data access, APIs, event handling, asynchrony, and limits of parallelism. These ideas are connected to familiar CS1 topics such as loops, arrays, searching, sorting, functions, objects, graphical interaction, data access, and basic algorithmic reasoning. Recurring activity families include unplugged activities such as Flag Maker and Penny Search/Penny Sorting; visualizations and simulations for parallel search, linked lists, flag coloring, and Zombie Attack; and lightweight programming activities involving OpenMP, remote earthquake-data APIs, Greenfoot or event-driven programming, image processing, and parallel search or sorting.

The chapter is organized as an adopter-facing guide. For each popular activity, it summarizes prerequisites, learning outcomes, materials, timing, classroom logistics, discussion prompts, teaching tips, and adaptation choices where available. It also highlights common implementation challenges, including limited CS1 instructional time, novice cognitive load, instructor and teaching-assistant preparation, software setup, activity logistics, class-size differences, and assessment constraints. The chapter points readers to institutional chapters, repositories, appendices, assessment instruments, student feedback, and classroom evidence so they can select and adapt materials that match their own teaching context.

Across institutions, shared lessons include starting with intuition, using unplugged or visual activities before code, keeping programming tasks lightweight, and embedding PDC concepts within existing CS1 learning goals rather than treating them as disconnected advanced content.

Suggestion for this Chapter (Vaidy):

It should probably have a different format than subsequent chapters. So the descriptor elements need not be there. In the abstract we can list the problems considered in this chapter and provide pointers to relevant sections. (see also in context of the comment in red on page 42).

The abstract should also list other contents of the chapter (Currently Sections 1.5 and beyond).

Sections 1.1 and 1.2, while important, will probably get in the way if someone is quickly trying to get to the problems themselves. They could be repositioned.

1.1 Introduction (Alan, Sushil)

[Motivation and background for the chapter.]

Purpose and role of Chapter 2

- common CS1 activities, challenges, implementation strategies, and lessons learned distilled across eight institutional exemplars: Chapters 2, 3, 4, 5, 6, 7, 8, and 9
- bridge from the project overview and roadmap in Sections 0.2 and 0.6 to the institution-specific CS1 chapters
- shared activity families and reusable guidance across TNTECH, Knox, USI, UNL, Webster, Casper, HPU, and MSU
- distilled entry point for adopters before richer local implementation details in the institutional chapters

Adopter-first orientation and infusion pathways

- single class activity, short module, lab assignment, or broader course-level CS1 infusion
- multiple CS1 entry points: unplugged activities, visualizations, animations, lightweight programming assignments, remote-data/API work, event-driven activities, performance comparisons
- instructor choice by language, class size, instructional time, student preparation, teaching support, and desired depth of PDC integration
- overall CS1 integration strategies summarized in Section 1.2

Core PDC ideas and CS1 connections

- accessible PDC ideas: data parallelism, task decomposition, speedup, load balance, communication, contention, distributed data access, APIs, event handling, limits of parallelism
- CS1 foundations reinforced (and evaluated): loops, arrays, searching, sorting, functions, objects, graphical interaction, data access, basic algorithmic reasoning
- PDC embedded within existing CS1 learning goals, rather than treated only as additional advanced content
- conceptual-to-programming progression: intuition first, code later

Common activity families

- unplugged activities: Flag Maker in Section 1.3.1; Penny Search/Penny Sorting in Section 1.3.2
- processor and workload modeling: data partitioning, speedup, load imbalance, contention, communication overhead
- visual and simulation activities: parallel search, linked lists, flag coloring, Zombie Attack, decomposition/coordination scenarios in Section 1.3.3
- plugged-in activities: Programming Flag Maker in Section 1.4.1; Greenfoot/event-driven work in Section 1.4.2; remote earthquake-data/API assignment in Section 1.4.4
- image-processing and parallel search/sort extensions as additional programming pathways

Challenges, lessons, evidence, and navigation

- common challenges: limited CS1 time, novice cognitive load, instructor/TA preparation, activity logistics, software setup, class-size differences, assessment constraints
- shared lessons: unplugged or visual intuition before code; lightweight programming tasks; PDC embedded in existing topics; clear timing, setup, and discussion guidance
- course-wide survey structure and common assessment context in Section 1.5
- institutional implementations, appendices, repositories, assessment instruments, student feedback, and classroom evidence as adoption supports
- reader navigation by activity type, programming language, class size, instructional time, and adoption goal

Chapter organization

Figure 1.1 shows how this chapter is organized.

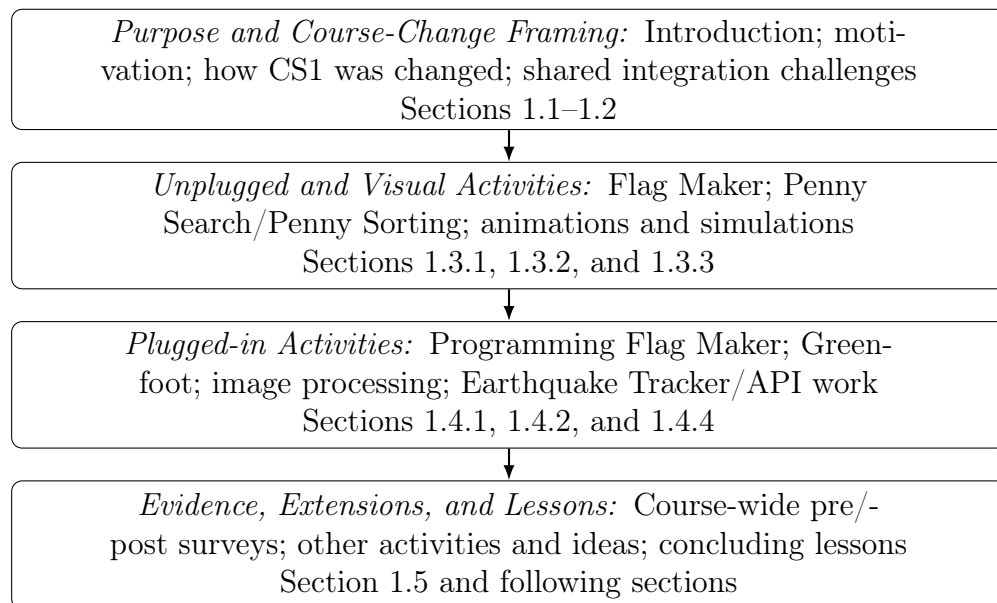


Figure 1.1: Organization of Chapter 1 – common CS1 activities and adoption-guidance.

1.2 How CS1 was changed and PDC topics integrated (April)

Across the participating institutions, parallel and distributed computing topics were incorporated into CS1 through three broad, and sometimes overlapping, strategies: selectively reducing existing content, modifying current assignments and activities, and adding short PDC-focused modules to the established course [1]. The institutions represented a wide range of instructional settings, including public and private universities, small liberal arts colleges, and larger research institutions. Their courses also differed in programming language, class size, contact hours, laboratory structure, and semester length. Because of these differences, the project did not prescribe a single model for changing CS1. Instead, each institution adapted the materials to fit its local curriculum while preserving the foundational programming knowledge expected of beginning students.

Some institutions created space by reducing or deferring topics considered specialized, redundant, or better suited for CS2. Examples included binary and random-access files, C-style strings and arrays, detailed output formatting, two-dimensional arrays, advanced class relationships, static members, garbage collection, or additional practice with object-oriented concepts. In most cases, these topics were not viewed as unimportant; rather, instructors determined that they could be covered more briefly or revisited in later courses. Other institutions retained the full CS1 syllabus and introduced PDC without removing content. One institution, for example, placed three asynchronous PDC modules outside regular class time, while another used existing laboratory and recitation periods to provide PDC activities without restructuring the lecture sequence. Across institutions, the amount of direct instructional time devoted to PDC ranged from no scheduled class time for asynchronous modules to approximately seven hours, generally representing one to two weeks of student engagement.

The most common strategy was to embed PDC concepts into topics already taught in CS1. Existing instruction on loops, arrays, searching, file processing, classes, objects, and problem decomposition was reframed to include ideas such as data parallelism, task decomposition, distributed data access, event handling, scalability, speedup, and computational cost. Examples included modifying array summation and searching assignments to discuss parallel execution, using serial and parallel versions of programs so students could compare execution times, accessing earthquake data through remote APIs, introducing OpenMP within an arrays laboratory, and using event-driven game programming to teach classes and objects. At other institutions, Flag Making 1.3.1, Penny Search 1.3.2, Minecraft, visualizations, animations 1.3.3, and other exercises were used to help students reason about parallelism before encountering implementation details. These changes generally layered PDC learning objectives onto traditional CS1 objectives rather than replacing them. Students still practiced loops, arrays, classes, file processing, and algorithmic thinking, but did so within contexts that reflected contemporary multicore, distributed, and event-driven computing.

The form of PDC instruction also varied according to institutional needs and instructor expertise. Some courses emphasized conceptual understanding through readings, discussions, animations, and codeless demonstrations. Others included guided programming exercises using OpenMP 2.4.2, Java parallel libraries, remote data sources, or event-driven

environments such as Greenfoot 1.4.2. Several institutions used unplugged activities in which students physically modeled processors, divided work, or compared serial and parallel approaches. This variety was intentional: the goal was not to make students proficient parallel programmers during their first course, but to establish an early awareness that computation is not always sequential and that modern problems may involve multiple processors, distributed data, asynchronous events, and performance tradeoffs. Taken together, the implementations demonstrate that PDC can be introduced through incremental and locally appropriate changes rather than through a complete redesign of CS1.

1.2.1 Challenges (April)

The most significant challenge was *finding instructional space in an already crowded CS1 curriculum*. Nearly every existing topic could be justified as useful, making decisions about what to condense difficult. The redesign therefore required instructors to distinguish between concepts that were essential for immediate student success and topics that were specialized, repeated in later courses, or could be introduced with less detail. Even when no topic was eliminated completely, reducing practice or lecture time carried the risk that students would receive less reinforcement of traditional material. Careful alignment among course outcomes, assignments, laboratories, and later courses was necessary to ensure that the revised curriculum did not create gaps in students' preparation.

A second challenge involved *presenting PDC at an appropriate level for novice programmers*. Students in CS1 are simultaneously learning syntax, control structures, debugging, and problem-solving strategies. Introducing threads, race conditions, synchronization, or implementation-level details too early could create unnecessary cognitive load. Consequently, the revised course emphasized conceptual understanding, visual demonstrations, bounded code examples, and unplugged activities before asking students to interact with parallel code. The intent was not for students to become proficient parallel programmers in CS1, but for them to recognize that computational work can be divided, understand that modern computers contain multiple processing units, and begin reasoning about the potential benefits and limitations of parallel execution.

The redesign also required substantial *faculty effort*. Developing age-appropriate examples, identifying suitable locations in the existing curriculum, revising assignments, testing software and datasets, and repeatedly refining the materials required more than 100 hours across the initial development year. Coordination with laboratory instructors and teaching assistants added another layer of complexity because those supporting the course needed enough familiarity with the new concepts and technologies to guide students effectively. Technical differences among student computers, compiler configurations, OpenMP support, and execution timing also had to be anticipated. Although adopting already-developed modular materials can substantially reduce this workload, successful implementation still requires local adaptation, instructor preparation, and continued refinement based on student responses.

Please read this in the context of the comments on page 38). with suitable adjustments the suggestion here may not be needed.

Add an initial list of problems considered (Flag making etc...) before the next section. For each element of the list add a short description and a pointer to relevant sections in this chapter and possibly others. This will help the reader move more quickly to what they really want to see (the detailed exemplars in subsequent chapters). Also if a person is coming from a later chapter to Chapter 2, we want them to be able to speedily get back to the later chapter.

1.3 New Unplugged Activities

1.3.1 Flag Maker (David, April)

Activity Summary

The Flag Maker unplugged activity was the most widely adopted activity created by our project. It has students take on the role of the processor to color cells of a grid in order to make the image of a flag. They do this sequentially and then in a variety of parallel configurations. The activity illustrates speedup and several concepts of coordination and contention in a graphical manner. It is a parallelized version of a graphical but non-parallel programming assignment to teach about loops by Emad [2].

This activity has been previously published [4]; the material on it here draws from that document.

Prerequisites

The prerequisites for the Flag Maker activity are minimal. It has been used successfully as early as 1/3rd of the way into CS1, but should be suitable even for a CS0 class. The students need to understand the idea of operations executing one at a time and how loops allow repetition, but they do not even need conditionals or functions. The activity is built on the idea of the computer being able to perform multiple simultaneous operations, but we found that we could just tie this to the terms “dual-core” and “quad-core”, with which the students were already familiar.

Learning Outcomes

Upon completing the activity, participants will be able to:

1. recognize parallel computing terms such as speedup, contention, and pipelining;
2. recognize algorithmic solutions that are representative of data parallelism;
3. describe how parallel execution time is bounded by the slowest processor (i.e., the critical path).



Figure 1.2: Flag of Mauritius [7]

Logistics

We split the class into groups of 5–6 or groups of 2–3, pairs of which merged to create groups of 5–6 after the first two phases of the activity. Each group needs 4 sheets of paper (1 per phase) with a large grid, each cell of which will be a single pixel in their drawing. (Information on the paper to use is in Section 1.3.1.) They also need one drawing implement of each of the flag’s colors (red, blue, yellow, and green). The most popular implement is thick markers, but thin markers, bingo daubers, colored pencils, or crayons can also be used; expect unhappy students if they are provided with pencils or crayons, though giving these to some groups is a pedagogic choice discussed below. Each group will also need a way to keep time; we had students use the stopwatch program on their phones.

Running the Activity

Initially, the instructor introduces the activity by telling students they will take the role of computer processors to color a flag. Introduce the flag of Mauritius (see Figure 1.2), an island nation off the east coast of Africa. Split the class into appropriate groups and pass out the materials. It is also helpful to clarify the expectations on coloring: each person should color a single grid cell at a time (not scribble across a whole row) and to fill in the cells consistently. Consistency is the main requirement here, but we tend to encourage a style where each cell is colored pretty thoroughly, but not perfectly. (Perfection will greatly slow down the activity, but you want something that looks like the flag when completed.) It is helpful to show the class a sample flag that you’ve already colored.

Then, the activity proceeds in four scenarios, each with a different configuration of students coloring the flag. These scenarios are depicted in Figure 1.3, where P1–P4 mark the cells to be colored by up to four “processors” in the scenario. In the first scenario, one student (P1) in each group colors all the grid cells, one color at a time, while another student times them. After all the teams are done, have them report the times and record them on the board.

In Scenario 2 (top right), the flag is split in half, with one student (P1) coloring the red and blue stripes while another student (P2) colors the yellow and green stripes. In addition, there is a third student timing them; the time should be the finishing time of whichever

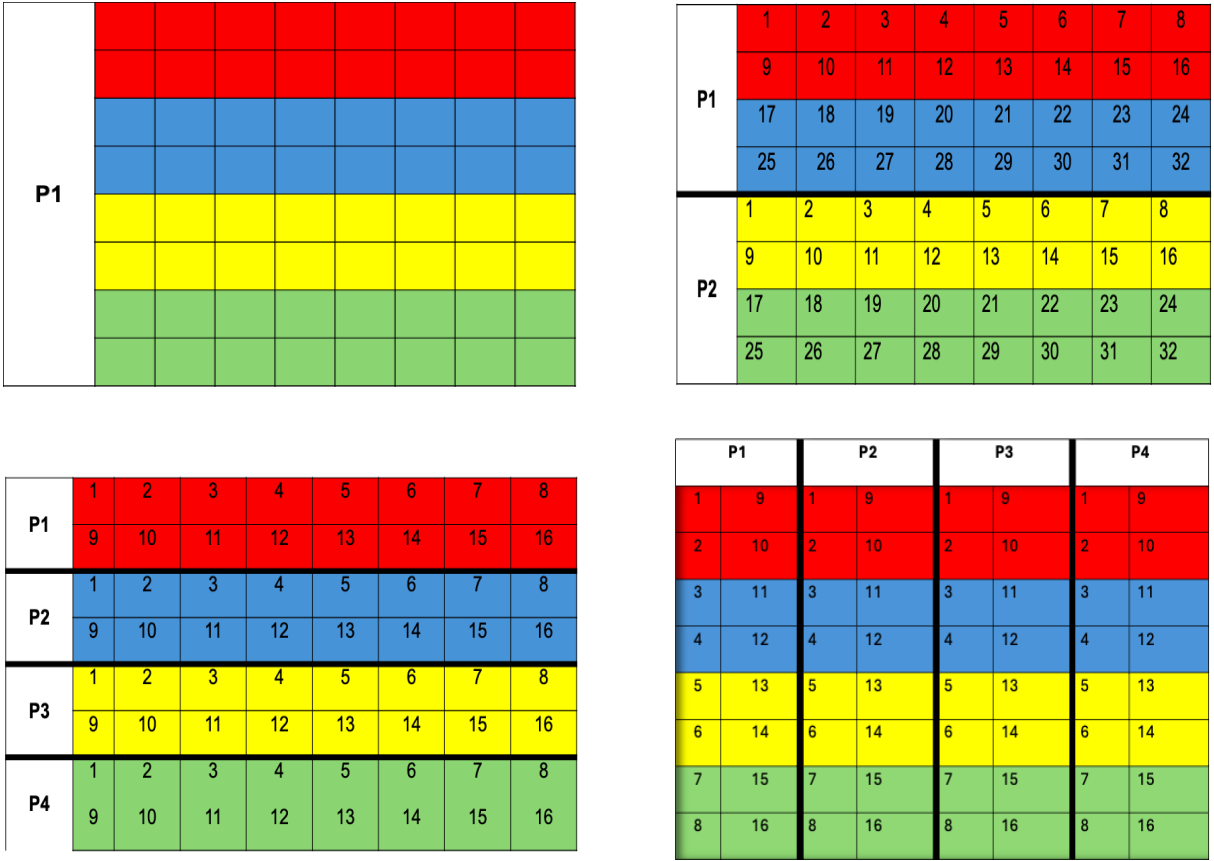


Figure 1.3: Graphical depiction of the Flag Maker scenarios: Scenario 1 (top left), Scenario 2 (top right), Scenario 3 (bottom left), and Scenario 4 (bottom right). P1–P4 depict the regions to be colored by each “processor”. The cells after Scenario 1 are numbered to show the order in which they should be colored.

student finishes last. After the coloring is completed, the times should again be recorded on the board.

Scenario 3 (bottom left) and Scenario 4 (bottom right) split the flag into 4 parts, each colored by one student, while a fifth student times them. Again, the time should be the finishing time of the last processor and the times should be collected on the board when all the groups have finished. In Scenario 3, each student colors one entire stripe. In Scenario 4, each student colors a vertical strip which includes two columns of each stripe. The cell numbering is also important here; each student is expected to color their first column, coloring red, blue, yellow, and green before returning to the top of their second column to repeat the colors. Note that each group should still have only one drawing implement of each color so the implements will have to be passed around.

Activity Discussion

Some institutions choose to discuss each scenario directly after each one, while others choose to have class discussion after the activity has ended. In both cases, the purpose of the discussion is to ask students what they observed and to put their observations into the context of parallel computing giving official terminology.

The first observation is likely that the coloring times decreased with the number of people doing the coloring. This leads to the definition of speedup (serial time divided by parallel time) and its calculation for each scenario. A particular term of interest is linear speedup, when the speedup is equal to the number of cores. More common is sublinear speedup, which occurs because the parallel program has to do extra work (like system calls to use all the cores) or some of them have to spend time waiting for a limited resource (like the markers in the activity). In parallel computing, extra work is overhead and resource-induced delays are contention. It is also possible to achieve superlinear speedup, one greater than the number of cores. In the activity, this may occur because students get more efficient with coloring as they practice or color less thoroughly as they get tired. Getting more efficient is somewhat like “warming up” a system, where a second run of the program can benefit from caching effects. In real systems, superlinear speedup is rare but can occur when performance is limited by memory rather than processing; running on multiple

Instructional Material

- TNTECH CS1 Flagmaker Presentation Slides
- TNTECH CS1 - Grids to Print for Activity

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- Knox CS1-Section3.4.1; Knox does post-activity discussion and has a Flag Maker plugged activity.

- TNTECH CS1- Section 2.3.2; TNTECH does discussion after each Scenario and has an OpenMP programming assignment 2.4.2 after the activity concludes.
- Webster CS1- Section 15.2.2

Experience and Teaching Tips

Data and Analysis

1.3.2 Unplugged Penny Search Activity (Srishti)

Activity Summary

The Unplugged Penny Sorting Search Exercise (PE) is an unplugged, hands-on active learning activity designed to introduce foundational concepts in parallel computing through a tangible, accessible task. Rather than relying on code or simulation, participants physically sort and search a collection of pennies to find the oldest one, enacting the roles of processors in a parallel system. The activity unfolds across three phases that progressively model a uniprocessor baseline, a balanced two-processor parallel execution, and a load-imbalanced five-processor scenario. By physically experiencing the difference in task completion time across these phases, participants develop an intuitive understanding of how parallel systems behave under varying conditions of data distribution and workload balance. Below is a more detailed description of the activity phases:

Phase 1: Uniprocessor phase At the data partitioning step, each group will receive a bag of pennies (e.g., 50), though the number can be reduced as long as the count remains an even number. In each group, Participant 1 performs sorting and searching for the oldest penny as their task on the allocated data (pennies), Participant 2 records time as their task and the other participants (if any) perform the task of observing. At a set time, all groups and group members begin their respective tasks.

- Participant 1 Task: find the oldest penny
- Participant 2 Task: tracks the time Participant 1 takes to complete the task and documents the results.
- Other participants: observe Participants 1 and 2. When they are done, shuffle coins and put them back in the bag.

Phase 2: Two Processor Parallel Phase At a set time, all groups will begin the tasks.

- Participants 1 and 2 of each group are given an equal number of pennies (e.g., 25). This workload or data distribution is managed by the instructor to create a load balanced parallel execution scenario.
- Participant 1 Task: Find the oldest penny in their bag.
- Participant 2 Task: Find the oldest penny in their bag.

- Participants 1 and 2 communicate and determine the overall oldest penny between both bags.
- Participant 3 Task: tracks the overall time for Participants 1 and 2 to determine the oldest penny in both bags.
- Any other participants: observe Participants 1, 2, and 3. When they are done, shuffle coins and put them back in the bag.

Parallel time is calculated as the time of the slowest participant among 1 and 2 and the communication time to determine the oldest penny of the two participants. When done observing, participants in the group shuffle coins and put them back in their respective bags. Each bag should have an equal number of pennies.

Phase 3: Five Processor Parallel Phase The activity facilitator or instructor gives each group four bags of two pennies for participants 1, 2, 3, and 4 and the remaining pennies (e.g., 42) to Participant 5. This uneven workload or data distribution introduces a load imbalance scenario for the parallel processes. At a set time, sorting begins for all groups.

- Participants 1-5 Task: Each must find the oldest penny in their bags. Once all participants complete their sorting, they must communicate with each other to identify the overall oldest penny.
- Participant 6 Task: tracks the overall time for Participants 1-5 to determine the oldest penny in their bags.
- Any other participants: observe Participants 1, 2, 3, 4, 5, and 6. When they are done, shuffle coins and put them back in the bag.

Parallel time is calculated as the time of the slowest participant among participants 1-5 and the communication time to determine the oldest penny of the five participants.

Prerequisites

This activity requires no formal prerequisites in parallel computing, making it suitable for introductory CS courses (CS1 and CS2) as well as upper-division systems courses where the instructor wishes to ground abstract concepts in concrete experience. Familiarity with basic algorithmic thinking, such as the idea that a search task has a measurable runtime, is sufficient preparation. The activity is also accessible to participants from non-CS disciplines participating in computing outreach or interdisciplinary courses.

Learning Outcomes

Upon completing the activity, participants will be able to:

1. explain the concept of data parallelism and how a dataset can be partitioned across multiple processors;
2. distinguish between load-balanced and load-imbalanced parallel workloads and articulate the performance implications of each;

3. describe how parallel execution time is bounded by the slowest processor (i.e., the critical path);
4. explain the concept of Amdahl's Law;
5. recognize the role of inter-processor communication in determining overall parallel performance; and
6. compare theoretical speedup with observed speedup from recorded timing data across the three phases.

Implementation Details

The PE activity is delivered as part of a self-contained instructional module on parallel computing. The module follows a deliberate pedagogical arc: it begins by motivating the relevance of PDC as a curriculum topic, transitions into conceptual grounding through short video resources, moves into the unplugged activity itself across three phases, and closes with structured group discussion and an optional survey. This arc is consistent whether the module is taught as a standalone guest lecture, a lab session, or an integrated class period within a broader CS1 or CS2 course.

The overall session outline is as follows: (1) motivation for why PDC belongs in the undergraduate curriculum, (2) an introduction to the concept of parallel computing through curated video resources, (3) the unplugged penny sorting activity across its sequential and parallel phases, (4) a group and class-wide discussion reflecting on observed results, and (5) an optional post-activity survey to capture student perceptions. Each component is described in detail below.

(i) Instructional Material (Lectures, Videos, Assignments, Tests, Discussions)

- **Motivation:** the session opens with a brief framing of why PDC is a required component of the undergraduate computing curriculum, grounded in the joint initiative mandated by the National Science Foundation (NSF) and the ACM/IEEE Computer Society. Pointing students to the PDC curriculum rationale (available at <https://grid.cs.gsu.edu/~tcpp/curriculum/>) situates the activity within a broader disciplinary context and helps students understand that parallel thinking is not an elective skill but a core competency expected of modern computing graduates. This framing takes no more than three to five minutes and significantly increases buy-in, particularly in courses where students may not have expected to encounter any PDC content.
- **Video resources:** before launching into the physical activity, a short video segment introduces the concept of parallel computing at an accessible level. Two videos are recommended for in-class use: a light, metaphor-driven introduction ("Digging Holes," available at https://youtu.be/bun_WSB9iRw?si=-1KANzuQwGat8dTW) and a more substantive technical overview (available at https://youtu.be/GXrI3oJ3Eak?si=c6VuvHoZ92_4JQQh). The LLNL parallel computing tutorial (https://computing.llnl.gov/tutorials/parallel_comp/) and Chris

Bourke’s webpage at University of Nebraska at Lincoln (<https://cse.unl.edu/~cbourke/PDCatUNL/module-parallel.html>) are recommended as post-class resources for students who wish to explore further. The two in-class videos together run approximately eight to twelve minutes and can be trimmed depending on available time. Their purpose is not to teach parallel computing comprehensively but to give students just enough conceptual vocabulary (processor, task, speedup, parallelism) to make sense of what they are about to enact physically.

- **Activity materials:** the primary instructional artifact is the activity itself, which requires minimal preparation overhead. One bag of pennies per group (approximately 50 pennies, though smaller or larger counts work if the count is even) is needed for Phases 1 and 3. For Phase 2, two bags per group containing an equal split (e.g., 25 pennies each) are required. Each group also needs a timer, and it is recommended that the instructor prepare an optional recording sheet on which groups log the elapsed time for each phase, the oldest penny identified, and any observations about coordination or idle time. This sheet doubles as a data collection instrument for the post-activity discussion. Note: if acquiring pennies is challenging, then instructors may also use custom index cards with random years printed on them, craft buttons with years printed on them using a permanent marker, or playing cards. April Crockett’s TNTECH instructor presentation slides that walk through the scenarios and printable lab sheets are available at https://github.com/cscaprilcrockett/CDER_TNTECH_CS1/tree/main/Unplugged%20Activities/Penny.
- **Assignments, tests, assessments:** To reinforce the concepts introduced through the activity, a short follow-up assignment or test can ask students to calculate the observed speedup from Phase 2 and Phase 3 using their recorded timing data, compare it against ideal (linear) speedup, and reason about the sources of deviation. Students can further be asked to connect their observations to Amdahl’s Law by estimating the parallelizable fraction of the task and quantifying the overhead introduced by inter-processor communication. An optional post-activity survey (such as the ASPECT survey [9]) is recommended for capturing student perceptions of the activity’s clarity, engagement, and perceived learning value. Survey data is particularly useful for instructors iterating on the activity across semesters or presenting the pedagogy at teaching focused venues.

(ii) How-To Guide

- Session structure: plan for at least a 50-minute class period. A suggested time allocation is: five minutes for PDC motivation, ten to twelve minutes for video viewing, twenty to twenty-five minutes for all three phases of the activity (including penny redistribution between phases), and ten to fifteen minutes for the group and class-wide discussion. If time is limited, the motivation segment and one of the two videos can be reduced without compromising the activity itself.
- Before class: prepare one bag of approximately 50 pennies per group for Phases 1 and 3, ensuring each collection spans a wide range of years so that the search is non-trivial. Prepare two additional bags per group with equal penny counts for Phase 2 (e.g., 25 each). Print or otherwise distribute the optional recording sheet. Confirm that each

group has access to a timer. If using the videos in class, test the links and audio in advance.

- Forming groups and running phases: divide the class into groups of five or six. Groups of four can participate in Phases 1 and 2 with minor adjustments but require at least five participants to enact Phase 3's load imbalance scenario meaningfully, since the contrast between participants with two pennies and the participant with the bulk of the coins is the core pedagogical mechanism of that phase. Run Phases 1, 2, and 3 according to the description provided above.
- Group and class discussion: after all phases conclude, facilitate a discussion at the group level first, then open it to the full class. Prompt students to share their recorded times, compare across groups, and articulate why Phase 3 did not improve on Phase 2 despite engaging more participants. Key observations to draw out include: the parallel runtime is bounded by the slowest participant; Participants 1 through 4 in Phase 3 were idle for most of the phase; and the act of comparing results across participants introduces communication overhead that is non-trivial even at small problem sizes. This is also a natural point to introduce or revisit Amdahl's Law formally.
- Assessments: conclude the session with a follow up assignment, test, or a post activity survey.

Experience and Teaching Tips

This activity runs effectively in a standard 50-minute or a 75-minute class period, with approximately 20 to 25 minutes devoted to all three phases and the remainder reserved for the post-activity discussion and assessments. In practice, Phase 1 tends to take the longest for individual participants and provides a useful baseline that makes the speedup in Phase 2 viscerally apparent. The dramatic slowdown (or absence of speedup) in Phase 3 due to load imbalance consistently generates spontaneous reactions from students, which makes the subsequent discussion highly engaging. One common facilitation challenge is that participants in the load-imbalanced phase (Participants 1-4) finish their two-penny assignments almost instantly and then sit idle while Participant 5 continues sorting. This idle time is itself a powerful teaching moment as students directly experience what it means for processors to be underutilized. Instructors should call attention to this idleness explicitly, as it connects naturally to the concept of processor starvation and the importance of effective scheduling. Timing precision varies across groups, and recorded times may differ more than expected from the theoretical speedup. Rather than treating this as a flaw, instructors can use the variance to discuss real-world sources of overhead: participants search at different speeds, communication takes time, and the act of comparing two pennies is not instantaneous. This mirrors the behavior of real parallel systems and reinforces why the theoretical speedup is an upper bound rather than a guarantee.

For larger class sizes, it is advisable to have a teaching assistant circulate across groups during Phase 2 and Phase 3 to ensure that the penny distribution is carried out correctly and that timing starts simultaneously across all groups. For very large lectures, the activity can alternatively be demonstrated at the front of the room with one volunteer group while the

rest of the class observes and predicts the outcome of each phase before the timer is stopped. Finally, instructors should allow participants to observe the timing data first and then derive the intuition themselves before the formal definition of Amdahl's law is introduced, as this leads to considerably stronger engagement and retention. The activity works best as a discovery experience, not an illustration of a concept already explained.

Data and Analysis

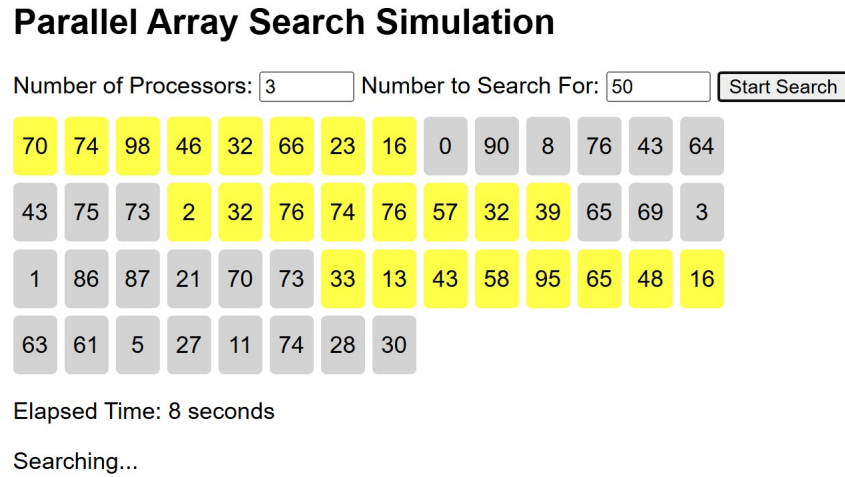
This activity was originally proposed by Mary Smith and Srishti Srivastava, with consultation from Brett A Becker [5]. In the initial phase of the development of this activity, its implementation was primarily evaluated through a student engagement survey based on the Wiggins et al. (2017) ASPECT survey[9]. the modified ASPECT survey, as presented in Figure 1.4.

Student Engagement Survey (Based on Wiggins, Eddy et. al (2017) Assessing Student Perspective of Engagement in Class Tool (ASPECT))				
Gender: Male	_____	Major: Computer Science	_____	
Female	_____	Engineering	_____	
		Other:	_____	
Instructions: The following questions ask you about your experience with the More Processors are Not Always the Best activity that you completed today. Please rate how strongly you agree or disagree with each of the following statements. Circle your response				
1) Explaining the material to my group improved my understanding of it.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
2) The instructor's enthusiasm made me more interested in the More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
3) Having the material explained to me by my group members improved my understanding of the material.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
4) Group discussion during the More Processors are Not Always the Best activity contributed to my understanding of the course material.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
5) The instructor put a good deal of effort into my learning for today's class.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
6) I had fun during today's More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
7) Overall, the other members of my group made valuable contributions during the More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
8) The instructor seemed prepared for the More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
9) I would prefer to take a class that includes this group activity over one that does not include this More Processors are Not Always the Best group activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
10) I am confident in my understanding of the material presented during today's More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
11) I made a valuable contribution to my group today.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
12) The instructor and TAs were available to answer questions during the More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
13) The More Processors are Not Always the Best activity increased my understanding of the course material.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
14) I was focused during today's More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
15) The More Processors are Not Always the Best activity stimulated my interest in the course material.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
16) I worked hard during today's More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree

Figure 1.4: Modified ASPECT survey used for the Penny Sorting unplugged activity

This survey evaluates three key constructs: the perceived value of the activity, instructor

Figure 1.5: Parallel Search Animation



contribution, and personal effort. The responses to each survey question were measured on a 5-point Likert scale, with values ranging from 1 (Strongly Disagree) to 5 (Strongly Agree). Faculty observational data were gathered during the students' participation in the activity.

The demographic variables, including gender, major, class standing, and age, were collected to explore possible relationships between these factors and engagement constructs. In summary, our analysis revealed that gender did not significantly impact the perceived value of the activity, instructor contribution, or personal effort across both examined activities. Age, however, showed a significant effect on instructor contribution in the PE activity. It is important to note that this initial analysis was conducted with a single group post-test and a small sample size, which may limit the generalizability of these findings. For a more in-depth understanding of this analysis, please refer to [5]. As the research evolved, the need to incorporate a pre- and post-survey approach is recognized to assess the extent to which the activities support student learning of key PDC concepts, such as concurrency, message passing, shared memory, scheduling, load balancing, speedup, and Amdahl's law.

1.3.3 Using Animations (Sue)

Parallel Search Animation

Problem Description Searching for a target value in a large collection is one of the simplest examples of an embarrassingly parallel problem. Similar to this idea: 1.3.2. This animation contrasts sequential and parallel search by allowing students to observe how dividing the search space among multiple workers can reduce execution time. See figure1.5. Students can adjust the number of workers and visualize how each worker is assigned a different portion of the dataset. The activity also introduces the idea that adding more workers does not always produce proportional speedup because of coordination overhead and workload imbalance.

The Animation can be found here: [Parallel Search Animation](#)

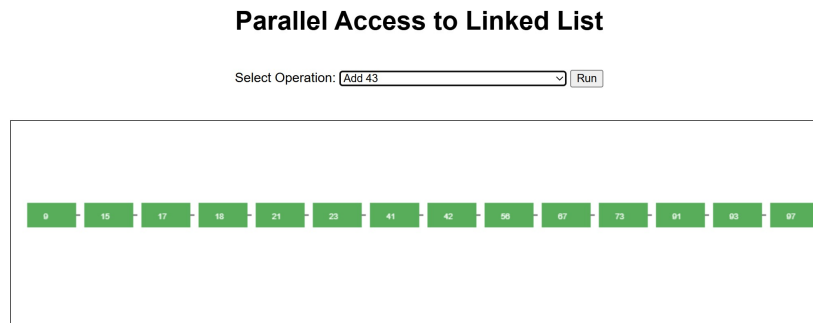


Figure 1.6: Parallel Linked List Animation

Prerequisites Students should understand basic loops, arrays (or lists), and linear search. No prior knowledge of parallel programming is required.

Learning Outcomes After completing this activity, students should be able to:

1. Distinguish between sequential and parallel search.
2. Explain how search tasks can be partitioned among multiple workers.
3. Describe why parallel search can reduce execution time.
4. Recognize factors that limit parallel speedup, including workload imbalance and synchronization overhead.

Parallel Linked List Animation

Problem Description Linked lists present a different set of challenges for parallel programming than arrays because elements are connected through pointers rather than contiguous memory locations. This animation demonstrates how multiple operations, including searching, inserting, and deleting nodes, can occur concurrently on a shared linked list. See Figure 1.6. Students can execute multiple operations simultaneously and observe how concurrent modifications may interfere with one another when proper synchronization is not used. The animation introduces fundamental concepts such as concurrent data structures, synchronization, race conditions, and mutual exclusion intuitively and interactively.

The animation can be found here: [Parallel Linked List Animation](#)

Prerequisites Students should understand basic linked list operations, including traversal, insertion, and deletion. A basic understanding of pointers or references is helpful, although no prior knowledge of parallel programming is required.

Learning Outcomes After completing this activity, students should be able to:

1. Explain why concurrent operations on linked lists require synchronization.

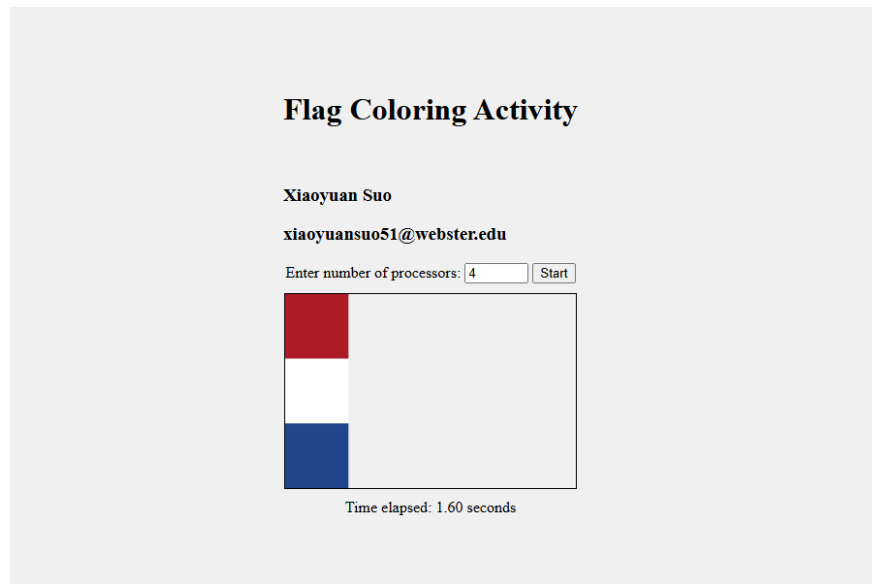


Figure 1.7: Flag Coloring Animation

2. Distinguish between read-only operations (search) and modifying operations (insert and delete).
3. Recognize how race conditions can arise when multiple threads access a shared linked list simultaneously.
4. Describe the role of mutual exclusion in maintaining data consistency during concurrent execution.

Flag Coloring Animation

Problem Description The Flag Coloring animation introduces task decomposition through an interactive coloring activity inspired by the classic map-coloring problem. Similar to the unplugged activity described in Section 1.3.1, students are asked to assign different workers to color independent regions of a flag while ensuring that adjacent regions do not share the same color. See Figure 1.7. As students assign work to multiple workers, they observe that many regions can be colored concurrently, whereas neighboring regions introduce dependencies that require coordination. The activity provides an intuitive introduction to task decomposition, concurrency, synchronization, and dependency analysis without requiring any programming experience.

The animation can be found here: [flag coloring animation](#)

Prerequisites No programming experience is required. The activity is suitable for CS0, CS1, CS2, or non-computing audiences. Students only need a basic understanding of following rules and solving simple logical puzzles.

Learning Outcomes After completing this activity, students should be able to:

Zombie Outbreak Simulation

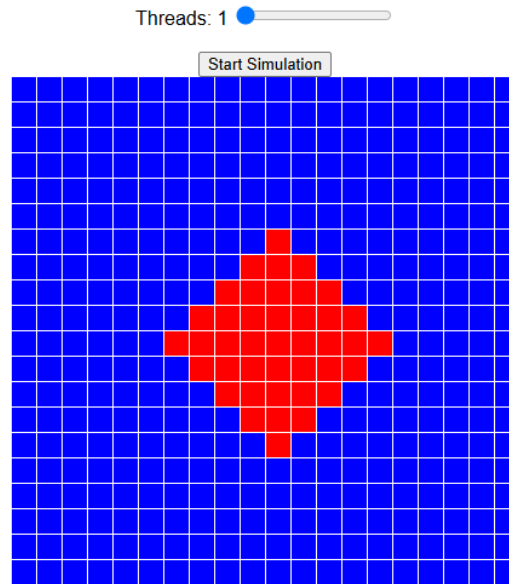


Figure 1.8: Zombie Attack Animation

1. Explain the concept of task decomposition by dividing a larger problem into smaller independent tasks.
2. Identify tasks that can be executed concurrently and tasks that require coordination because of dependencies.
3. Recognize that not all work can be performed in parallel due to shared constraints.
4. Relate graphical task decomposition to the design of parallel algorithms and workload distribution.

Zombie Attack Animation

Problem Description The Zombie Attack animation introduces parallel computing concepts through a simulation of a zombie outbreak spreading across a two-dimensional grid. Similar to the programming activity described in Section ??, each cell in the grid represents an individual who may be healthy or infected. During each simulation step, infected individuals may spread the infection to their neighboring cells. See Figure 1.8. Students can divide the grid among multiple workers and observe how different regions are processed concurrently while neighboring partitions must exchange information at their boundaries. The activity provides an intuitive introduction to domain decomposition, parallel simulation, synchronization, and communication between neighboring tasks.

The animation can be found here: [Zombie Attack animation](#)

Prerequisites Students should understand basic loops, conditional statements, and two-dimensional arrays (or matrices). No prior knowledge of parallel programming is required.

Learning Outcomes After completing this activity, students should be able to:

1. Explain how a two-dimensional problem can be partitioned into smaller regions for parallel execution.
2. Describe the concept of domain decomposition and its role in parallel simulations.
3. Recognize that neighboring partitions must communicate to correctly update boundary cells.
4. Explain how synchronization ensures that all workers complete one simulation step before advancing to the next.
5. Relate the simulation to real-world parallel applications such as disease modeling, wildfire spread, weather forecasting, and image processing.

Experience and Teaching Tips for All the Animations

More discussions can be found in my previous conference publication: [6]

We found that animations were an effective way to introduce abstract PDC concepts before students encountered programming implementations. Compared with traditional unplugged activities, animations required substantially less classroom time while still promoting discussion and conceptual understanding. Students generally found the visualizations engaging because they could immediately observe the effects of changing parameters, such as the number of workers, workload distribution, or decomposition strategy.

Animations worked particularly well when integrated with short instructor-led discussions rather than being presented as standalone demonstrations. Instructors were encouraged to pause the animation periodically and ask students to predict the outcome before continuing the simulation. These prediction activities often revealed common misconceptions about parallel execution and provided opportunities for immediate feedback.

Although the animations are designed to be self-contained, they are most effective when followed by a simple programming exercise or classroom discussion that connects the visualization to actual implementations. Instructors should also emphasize that animations intentionally simplify many aspects of parallel execution in order to highlight the underlying concepts rather than the implementation details.

Data and Analysis

Survey data were analyzed using descriptive statistics to assess changes in students' perceptions and understanding of parallel computing concepts. Specifically, Likert-scale responses from the post-activity survey were summarized to gauge the perceived effectiveness and engagement of the unplugged activity.

Key findings from the post-activity survey include:

How helpful was the activity in learning following concepts?

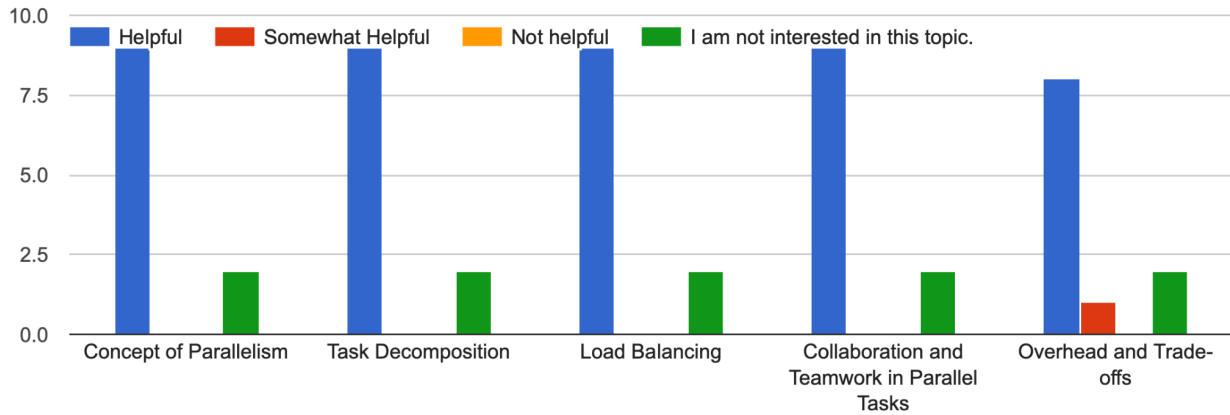


Figure 1.9: Responses from participants

- 90% of participants agreed that the activity reinforced their learning and helped them understand how parallel computing is applied in real-world contexts.
- 81% of participants agreed that the flag coloring activity increased their interest in parallel computing.
- 81% of participants indicated a preference for classes that incorporate group activities over those that do not.
- 90% of participants agreed that the activity was helpful in understanding key concepts such as parallelism, task decomposition, load balancing, collaboration and teamwork in parallel tasks, and overhead and tradeoffs.

Reflection responses were thematically coded to identify recurring themes related to conceptual understanding, engagement, and perceived value of the activity. The submissions to the Python parallel programming exercises were evaluated on the basis of precision, correct application of the parallel constructs, and clarity of implementation. Classroom observation notes provided additional qualitative evidence of student participation and collaborative interactions during the activities.

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- Webster CS1- Section ??

I feel prepared to apply parallel computing concepts to new problems beyond this activity.

11 responses

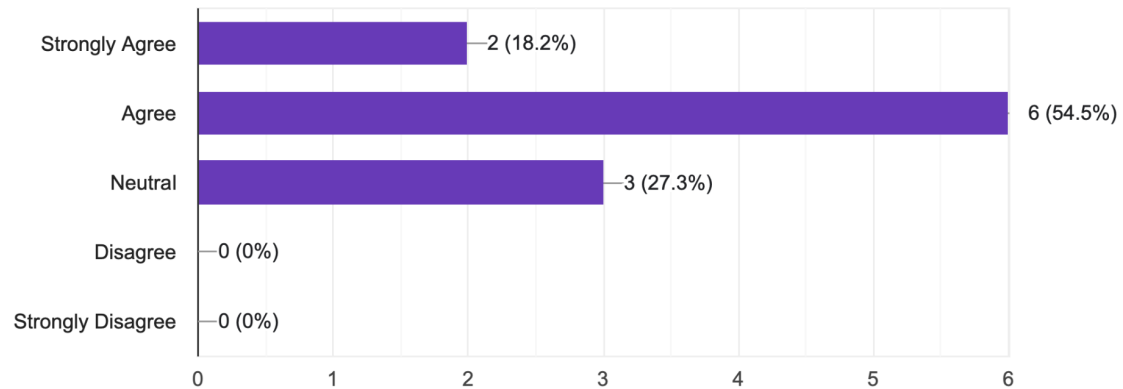


Figure 1.10: The students were then asked if they felt prepared to apply parallel computing concepts to new problems beyond this activity. 18.2 % strongly agreed with it. 54.5% agreed and 27.3 % were neutral.

1.4 New Plugged-in Activities

1.4.1 Programming Flag Maker

Problem Description, Prerequisites, and Learning Outcomes

The Flagmaker assignment teaches nested loops by asking students to write code that draws flags on a 2D grid. For example, the code in Figure 1.11 demonstrates one way to draw the Lithuanian flag, while Figure 1.12 shows the visualization generated by Flagmaker.

```
for (int r=0; r<grid.getNumRows()/3; r++)
{
    for (int c=0; c<grid.getNumCols(); c++)
    {
        grid.setColor(r, c, Color.YELLOW);
        grid.setColor(r + R/3, c, Color.GREEN);
        grid.setColor(r + 2 * R/3, c, Color.RED);
    }
}
```

Figure 1.11: Example C program displayed as verbatim text.

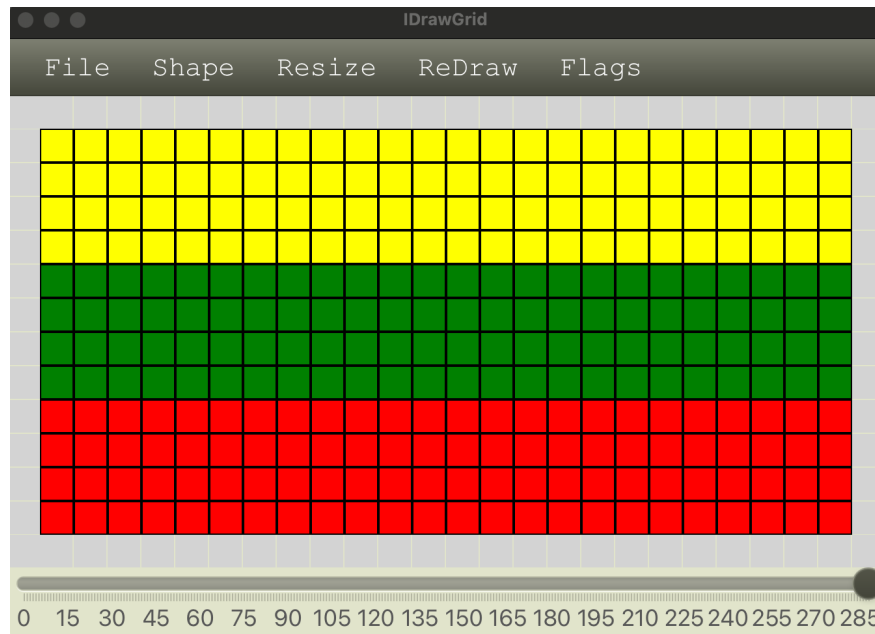


Figure 1.12: The Lithuanian flag, as drawn by the Flagmaker assignment.

Algorithms

Implementation Details

Autograder doesn't include code, just text files.

The Flagmaker assignment is based on an assignment from the mid-2000s originally created by Fawzi Emad, now a senior lecturer at the University of Maryland at College Park [2]. The original assignment, written in Java, used Java Swing. The current version uses JavaFX, which has pros and cons.

In general, demand for desktop GUI applications written in languages like Java is shrinking because many desktop apps have been replaced by control panels that connect to cloud-based SaaS (Software as a Service) backends. For those applications, HTML, CSS, and JavaScript are the dominant medium for structuring and styling text-heavy dashboard applications with standard input/output widgets (radio buttons, text inputs, checkboxes, etc). There are now tech stacks that build desktop applications with raw HTML, CSS, and JavaScript, such as Electron, which bundles in 150-200 Mb of Chromium and Node.js to handle rendering, or Tauri, which uses the platform's native browser for rendering. Thus, while we use JavaFX for the implementation, the students do not see any of these details, and we keep as much of this hidden from students as possible.

Styling desktop apps in Swing is incredibly tedious because Swing has no equivalent to CSS and so styling must all be done programmatically; meanwhile, JavaFX can be styled with a flavor of CSS, but it differs from regular CSS in enough ways that it's unclear that it is worth learning outside of the specific use cases where a Java desktop GUI application is required.

- **Instructional Material (Lectures, Videos, Assignments, Tests)**

The core Flagmaker assignment asks students to implement a variety of different flags using nested for loops. The assignment should provide students with a picture of the actual flag, and a picture of what we expect their code to generate. This is necessary, as we want students to simplify complex flags in the same way each time. For example, in Figure 1.13, we show the actual Japanese flag, as well as how we expect students to approximate the flag of Japan.

The core Flagmaker assignment includes an "autograder" feature that runs each student's flag code for a variety of valid sizes for that flag, and compares these outputs to the correct output. We don't ship any code to produce the correct flag output programmatically with the starter code for the students; instead we include the solutions as text files that the autograder reads into an internal format using library code that is hidden from the students. This is not a perfect solution, but so far has worked well enough in practice.

For example, we include a text file such as what is shown in Figure 1.14; The Flagmaker code reads this file into an internal format, and then runs student code for the same size to produce the same internal format, and then the autograder diffs them and includes it in a report for faculty or TAs to read in order to speed up grading.

- **How-To Guides**

Our main how-to guide is an 18 minute YouTube video available here: <https://youtu.be/y1tbzNnICz0>.

Experience and Teaching Tips

The most important tip is to change the flags every term! If we don't change the flags, previous versions of the assignment will be submitted. Thus far, we have found that desperate students will submit a copy of an entire assignment from a previous term, but are less likely to get access to copies of a previous version of the assignment, read through it, and copy only the code for specific flags. Thus, some flags can be re-used from term to term, so long as the assignment is not identical to the previous several terms.

We provide infrastructure to automatically generate the assignment each term, though it's typical behind the scenes academic code that needs some cleanup work. Please contact the authors from Knox College with any questions.

The current implementation of Flagmaker uses JavaFX, which is no longer part of regular JVM releases. Instead, JavaFX is released through OpenJDK along with each JVM release. Since JavaFX is not part of the JVM, it requires different libraries on Windows, Mac Intel, Mac with Apple silicon, and Linux. For experienced developers, this is one setting in a pom.xml or build.gradle file; for novices we avoid using Maven or Gradle whenever possible and instead use the run button in VS Code.

For Flagmaker with JavaFX, we provide copies of the different libraries, and students copy the appropriate library into the lib folder in VS Code. Every year, some students don't read the directions or pay attention to instructions, and then copy the wrong library, or don't copy the library, or copy the library into the wrong place, and their code doesn't compile or run. If the assignment does not run correctly, faculty or TAs should first ask: Did you copy the libraries correctly?

Graders need to actually look at the output from the autograder, and also at the code.
Can AI do the assignment? Probably with good prompting.

Data and Analysis

Do we have anything like this?

1.4.2 Greenfoot

Problem Description, Prerequisites, and Learning Outcomes

Greenfoot [3] [8]

(what is this paper about and how are our efforts different?) talk about event-driven programming
works for C++
we have videos

Algorithms

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)**

* Vague, under-specified assignments in order to get divergent assignments * avoid snake game and flappy bird – too many versions online

Instructional materials (what features does each one include?)

- <https://youtu.be/ITJR8ZaUvxQ> Hedgehog
- <https://youtu.be/LOu4fKjHvqo> Shooting Game
- https://youtu.be/p4Wec0k_9v4 Cards
- <https://youtu.be/dFJ-jdeyE7g> Zelda

- **How-To Guides**

Experience and Teaching Tips

Data and Analysis

1.4.3 Image Processing

Problem Description, Prerequisites, and Learning Outcomes

Algorithms

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)**

- **How-To Guides**

Experience and Teaching Tips

Data and Analysis

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- Webster University, Section 16.7
-

1.4.4 Earthquake Tracker Programming Assignment (April)

Problem Description, Prerequisites, and Learning Outcomes

The *Earthquake Tracker programming assignment*, asks students to complete a C++ program that retrieves real, live earthquake data from the United States Geological Survey (USGS), filters that data, and displays selected earthquake information to the screen. Unlike a traditional introductory programming assignment focused only on loops, functions, and basic syntax, this assignment introduces students to the broader idea of working with remote, distributed, live data through an API. Students use a USGS earthquake data feed, returned in JSON format, to see how programs can interact with real-world data sources and support socially meaningful applications such as natural disaster monitoring.

The *prerequisites* for this assignment is that students should already have experience with several foundational C++ concepts, including for loops, arrays, functions, Makefiles, JSON, cURL, and distributed data. Students are not expected to build an API or JSON parser from scratch; instead, they are given starter code, supporting libraries, and Makefile templates so they can focus on understanding how data is retrieved, parsed, filtered, and displayed.

The main *learning outcomes* are that students will be able to develop and implement an algorithmic solution for interacting with a remote service, practice working with JSON-formatted data, and compile/run a program using a Makefile. By the end of the assignment, students should understand the client-server relationship at a basic level: their computer acts as the client, sends a request to a remote USGS server through an API, receives live earthquake data, and then processes that data locally in a C++ program. A secondary learning outcome is conceptual. Students are asked to explain terms such as remote data, distributed data, API, and JSON, and to describe another real-world situation where large amounts of remote data are used to provide a public service. This documentation component reinforces the idea that computing is not only about writing isolated programs, but also about building systems that interact with live data sources in meaningful contexts.

Algorithms

The core algorithm for this assignment follows a retrieve–parse–filter–display structure. First, the program retrieves earthquake data from the USGS monthly earthquake feed. The

data source is a live GeoJSON file containing detected earthquakes from the past month. Because the data is dynamic, the exact output may differ from one run to another depending on when the program is executed.

Second, the program parses the returned JSON data and accesses the features array. Each element in this array represents an earthquake event. For each earthquake, the relevant information is stored inside the properties object, including values such as magnitude, place, timestamp, and tsunami indicator.

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- TNTECH - CS1 course - Section 2.4.1
-

1.5 Course-wide Pre and Post Course Surveys (Chris)

Development and testing teams administered the following common pre/post survey for CS1. The survey asked students their background knowledge in computing topics, programming languages, computing concepts, and PDC concepts. Students were asked to rate their prior experience on a 7-point Likert scale:

1. No Experience At All
2. Very Little Experience
3. Slightly Experienced
4. Between Slightly/Moderately Experienced
5. Moderately Experienced
6. Very Experienced
7. Extremely Experienced

The survey consisted of the following 26 items.

- | | |
|---|---|
| 1. Computer use experience (internet searches, email, discord, discussion groups, games, word processing, spreadsheets, organizing files & folders, etc.) | learning and/or formal programming course) |
| 2. Programming Experience (self-initiated | 3. Programming Experience compared to your classmates |
| 4. C | |

- | | |
|---|--|
| 5. C++ | 18. Calling functions/methods |
| 6. C# | 19. Arrays |
| 7. Java | 20. Writing static functions/methods |
| 8. JavaScript | 21. Writing functions/methods |
| 9. PHP | 22. Writing classes |
| 10. Python | 23. Creating objects |
| 11. Scratch | 24. Parallel processing: Computing where the operations within a program are being executed simultaneously (i.e. running on different cores of the processor at the same time) |
| 12. Visual Basic | 25. Distributed computing: Computing with a program that runs using data from a different computer (i.e. an on-line service) |
| 13. Other (any) | 26. Event handling: Having a function that responds to an independent action (i.e. a mouse click) |
| 14. Data/Variable Types (integers, doubles, char etc.) | |
| 15. Arithmetic Expressions and Operators | |
| 16. Branching (if/else, relational and logical operators, relational expressions) | |
| 17. Loops (while, for, etc.) | |

1.6 Other Activities and Ideas

[Alternative ways to apply these concepts, etc..]

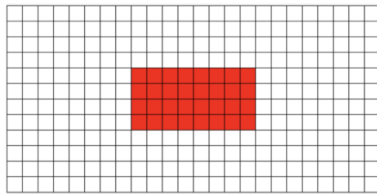
1.7 Conclusion

Japan



The flag of Japan has a white background with a red circle in the middle. Drawing circles is very difficult, so we will approximate the circle with a rectangle. Your rectangular red dot should be in the middle third of the flag. The height and width of the flag must be a multiple of 3.

Size 12



Size 24

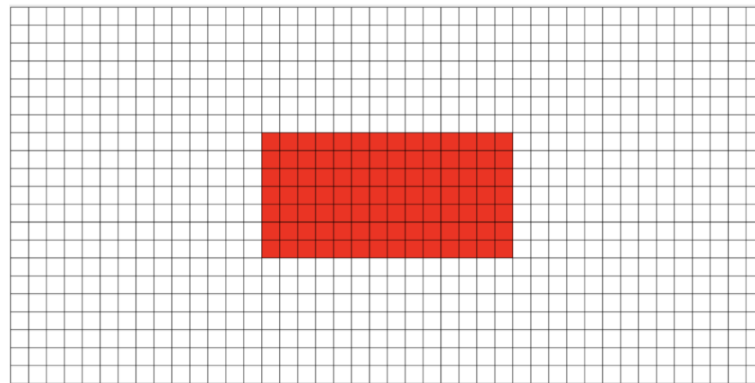


Figure 1.13: The assignment piece for the Japanese flag.

```

YYYYYYYYYYYY
YYYYYYYYYYYY
GGGGGGGGGGGG
GGGGGGGGGGGG
RRRRRRRRRRRR
RRRRRRRRRRRR

```

Figure 1.14: Solution as text for the Lithuanian flag.

Bibliography

- [1] April R. Crockett, Gerald C. Gannod, Xiaoyuan Suo, Chris Bourke, Mary L. Smith, Srishti Srivastava, David P. Bunde, Jaime Spacco, Jiayin Wang, Michelle Zhu, Neena Thota, Charles C. Weems, R. Vaidyanathan, Alan Sussman, and Sushil K. Prasad. Making room for parallel and distributed computing in CS1: approaches, tradeoffs, and faculty effort. In 2026 Frontiers in Education (FIE), pages –. IEEE, 2026.
- [2] Fawzi Emad. Flag coloring programming assignment. Personal communication to J. Spacco.
- [3] Michael Kölling. The greenfoot programming environment. ACM Trans. Comput. Educ., 10(4), November 2010.
- [4] M. Smith, S. Srivastava, D.P. Bunde, A. Crockett, M. Gerten, P. Maher, J. Spacco, X. Suo, J. Wang, and M. Zhu. A visual unplugged activity to introduce PDC. In Proc. Workshop on Education for High-Performance Computing (EduHPC), 2025.
- [5] Srishti Srivastava, Mary Smith, Amrita Ghimire, and Shaun Gao. Assessing the integration of parallel and distributed computing in early undergraduate computer science curriculum using unplugged activities. In 2019 IEEE/ACM Workshop on Education for High-Performance Computing (EduHPC), pages 17–24. IEEE, 2019.
- [6] Xiaoyuan Suo and Timila Dangol. Engaging First and Second Year Undergraduates with Parallel and Distributed Computing: Lessons from Unplugged and Game-Based Activities, pages 104–114. 06 2026.
- [7] Wikipedia user Zscout370. Flag of mauritius. https://en.wikipedia.org/wiki/Flag_of_Mauritius#/media/File:Flag_of_Mauritius.svg, viewed Jun 19, 2026.
- [8] Tamar Vilner, Ela Zur, and Shay Tavor. Integrating greenfoot into cs1: a case study. In Proceedings of the 16th Annual Joint Conference on Innovation and Technology in Computer Science Education, ITiCSE '11, page 350, New York, NY, USA, 2011. Association for Computing Machinery.
- [9] Benjamin L. Wiggins, Sarah L. Eddy, Lauren Wener-Fligner, Katherine Freisem, Daniel Z. Grunspan, Elli J. Theobald, Jessica Timbrook, and Alison J. Crowe. ASPECT: A Survey to Assess Student Perspective of Engagement in an Active-Learning Classroom. CBE—Life Sciences Education, 16(2):ar32, Summer 2017.

— 5. ATTACH APPENDIX —

Appendix

1..1 Vaidy Testing Section To Be Removed Later

This section was added by Vaidy to show us how to do different things.

I am testing the macros for the Bloom taxonomy. We we will use Applying (AP) for applying, for comprehending, and represent knowing by .

Bloom test

Observation 1.1 *This macro has two parameters. The first, “macro” is the label I chose. The second parameter (of which this text itself is a part) is the body of the observation.*

You can refer an observation, for example, Observation 1.1 and indicate its page number added as far away as follows: page 67.

This is an example of an inline comment. It can be as long as you want.

[illegible]

It cannot be split across paragraphs but newlines and spacing can be added.

Adding Index terms: The common ways are as follows. Their effect of the statements can bve seen at the end of the eBook:

- Here the term “interesting” is being indexed.
- As an “Extremely interesting” subcategory of “interesting,” you can use
- More information at <https://www.overleaf.com/learn/latex/Indices>.